

DOM: Test Strategy

Last updated by | Jose Casal | May 6, 2026 at 10:06 AM CST

**

Project: Azure-Based Data Ingestion Platform
Owner: Jose R Mora Casal
Date: October 2025

1. Introduction and Scope

This document outlines the comprehensive Test Strategy for the DOM project, a mission-critical initiative to automate and optimize the ingestion of structured and unstructured data into SQL databases. This strategy validates the solution's compliance, performance, and functionality across its cloud-native architecture.

1.1. Scope of Testing

Testing is end-to-end (E2E), covering all architectural layers from infrastructure provisioning to the user interface:

Layer	Technology	Scope Detail
Infrastructure	Terraform	Validation of IaC modules for provisioning Azure subscriptions, VNets, storage, and compute resources.
Data Pipelines (ETL)	Databricks	Verification of data quality, schema evolution, and complex data transformation logic.
Backend Services	.NET APIs	Functional, security, and performance testing of all RESTful endpoints.
Frontend	React	E2E functional verification, regression, and unit/component testing.
Authentication	Azure AD SSO	Validation of the single sign-on flow and Role-Based Access Control (RBAC) integrity.
DevOps Pipelines	ACI YAML	Validation of CI/CD pipeline correctness, security scanning, and automated deployment smoke tests.

2. 🎯 Test Objectives

The primary goals of the QA effort are defined by these measurable objectives:

- **Functional Correctness:** Achieve 95%+ pass rate on all documented functional test cases.
 - **Security Validation:** Verify the integrity and consistency of SSO and RBAC implementation across all environments.
 - **Data Integrity:** Validate data quality rules, schema conformity, and the accuracy of all transformation logic within Databricks pipelines.
 - **Performance:** Confirm API response latency and pipeline throughput meet documented non-functional requirements (NFRs) under peak synthetic load.
 - **Infrastructure Assurance:** Confirm Terraform module correctness and validate that production resources are provisioned according to the approved baseline.
 - **Automation:** Establish and maintain high-coverage automated test suites integrated into the CI/CD workflow.
-

3. 📌 Assumptions & Constraints

3.1. Assumptions

- **Environment Isolation:** Each client will utilize isolated Azure subscriptions and databases.
- **Test Data:** Synthetic data will be utilized in all non-production environments to ensure privacy compliance and repeatable test execution.
- **Auth Configuration:** Azure AD SSO and RBAC will be consistently configured for all environments (Staging, Production).

3.2. Constraints

- **API Testing:** API testing and automation will rely on defining contract tests and workflows exclusively via Postman/Newman.
 - **Tooling:** Testing is strictly limited to native tools for infrastructure (Terraform) and pipeline validation (e.g. Pytest, Great Expectations).
 - **Manual Effort:** Manual testing is a required, non-trivial component for initial requirement verification and in-depth exploratory testing.
-

4. 🧪 Test Coverage and Implementation

4.1. Detailed Test Case Creation

A core focus of the QA effort will be the detailed creation of test cases to ensure implementation validation and clear result recording:

- **Integration Testing:** Dedicated test cases will be engineered to validate inter-service communication and data flow paths. This includes calls between the React Frontend and .NET Backend, as well as the initial trigger mechanism from the Backend to the Databricks pipelines. Results will be recorded in Azure Test Plans.
- **Data Validation Testing:** Specific test cases will be written to validate data at different stages of the ETL process (Source, Staging, Target). These cases will cover:
 - **Schema Conformity:** Verifying all required fields are present and correctly typed.
 - **Data Quality Rules:** Testing specific business rules (e.g., uniqueness, completeness, validity).
 - **Transformation Logic:** Validating the output of complex Databricks logic against expected results.
- **Frontend & Backend Requirements (Functional Testing):** All core functional requirements (user workflows, CRUD operations) will have corresponding manual test cases created. Manual QA will execute these cases, and the results (Pass/Fail) and supporting artifacts (screenshots, logs) will be meticulously recorded in Azure Test Plans test runs.

4.2. Technology Stack Coverage

Layer	Technology	Test Types	Tools/Frameworks
Infrastructure	Terraform	Module validation, drift detection	terraform validate, terratest
Data Pipelines	Databricks	ETL, Schema/Data Quality	Manual Testing
Backend APIs	.NET REST	Functional, Security, Performance	Postman/Newman, OWASP TBD
Frontend	React	Unit Testing, E2E, Regression	Vitest, React Testing Library, Manual Testing, Playwright
Authentication	Azure AD	SSO, RBAC verification	Manual Testing, Playwright
DevOps	ACI YAML	Static analysis, Security Scans	Native Pipeline Validation

5. Automation Strategy

Automation is integral to reducing cycle time and maintaining regression coverage.

- **CI/CD Integration:** All automated tests (Unit, API, E2E) are triggered on merge via ACI pipelines.
- **Infrastructure:** Infrastructure validation is executed using native Terraform commands (plan, validate) and augmented by terratest for integration tests against the cloud environment.
- **Frontend E2E:** Playwright is the recommended framework for critical user journeys and regression suite execution.

6. Environment and Data Strategy

6.1. Environment Strategy

A multi-tiered, isolated environment structure is employed:

Environment	Purpose	Provisioning	Authentication
Dev/QA	Daily builds, component testing, initial manual verification.	Automated (Terraform)	SSO/RBAC (Test Credentials)
Staging	Formal regression, performance testing, Client UAT.	Automated (Terraform)	SSO/RBAC (Production Mirror)
Production	Live system deployment.	Automated (Terraform)	SSO/RBAC (Live)

6.2. Test Data Strategy

- **Source:** Synthetic data is mandatory for all non-production environments to ensure repeatable scenarios and eliminate PHI exposure.
- **Anonymization:** Strict data anonymization techniques are applied if any minimal real-world data is used.
- **Isolation:** Client environments are physically isolated to prevent cross-contamination during execution.

7. Risks and Mitigation

Risk	Impact	Mitigation Strategy
Data transformation errors	Data inconsistency, client reporting failures.	Implement robust schema validation within the Databricks pipelines.
SSO/RBAC misconfigurations	Unauthorized data access, security failure.	Include dedicated, automated authentication tests in every environment and as part of CI/CD gate checks.
Infrastructure drift	Resource misconfiguration, unexpected cloud costs.	Periodic validation of the current infrastructure against the IaC repository using Terraform validation tools.

8. Reporting, Metrics, and Approval

8.1. Reporting and Defect Management

- Test Cases & Execution: Managed exclusively in Azure Test Plans.
- Defect Tracking: Defects are logged and tracked through their full lifecycle in Azure DevOps.
- Visibility: Pipeline reports and dashboards provide CI/CD integrated reporting on test results, coverage, and stability.
- Alerts: Automated alerts are configured to trigger immediately on test failures (API, E2E) or security violations, directed to the QA and DevOps teams.

8.2. Approval and Sign-off

- Test Plan Ownership: Jose R Mora Casal, Owner.

Final Approval: The Client provides final approval for the production release after successfully completing UAT.